

המהדר קורא את הקוד שלך

שבוע: 12 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 13

משקל בציר הסופי: מטלת כיתה 12 · חלק א · חמישים אחוז מציון המטלה · 10 הטובות מתוך 12 נסמנות בשבוע: 40 דקות

מטרה

להעביר שכבה אחת — שכבת הרשת — לטיפוסים, **בלי לשנות שום דבר שהיישום עושה**, ולצאת עם משפט אחד שאפשר לבדוק: הרצת המהדר, והוא מצא הנחה שגויה על התשובה של מישהו אחר לפני שהרצת שורה.

בשבוע שעבר סגרתי את השיעור במשפט: **"js/api.js הוא המקום שבו ניחוש על מבנה של מישהו אחר הופך לקריסה, ובשבוע הבא המהדר תופס אותה לפני ההרצת"** היום גובים את זה, על אותו קובץ בדיוק.

זו לא כתיבה מחדש. **ההעברה הזאת היא שינוי משמר-התנהגות** — בדיוק המונח מהחצי השני של ההרצאה — ויש לה בדיקה אחת שסוגרת אותה: **diff** בין הפלט שנוצר לבין הקובץ של שבוע שעבר. מה שאמור להיות שונה שם הוא סוגריים ורווחים. **לא תנאים**. וזו גם המטלה שמסמנת V על דרישה 22–24 של הפרויקט, שמוגש בסוף השבוע.

תוצרי למידה

- לדעת מה טיפוסים תופסים ומה לא — ולומר את שני הצדדים.
- להבחין בין `unknown` ל-`any`, ולדעת למה `as` אינו בדיקה.
- לכתוב שומר בזמן ריצה שמצמצם (`value is T`), ולא רק מחזיר `boolean`.
- להקליד שאילתת DOM ומטפל אירוע בלי לפזר `as HTMLInputElement` בארבעים מקומות.
- להריץ `tsc` עם `tsconfig` מינימלי, ולהבין מה `strict` באמת קונה.
- לעבור מ-`js` ל-`ts`. **בהדרגה**, מודול אחד, בלי לעצור את שאר היישום.

מה יש בתיקייה

קובץ	מה עושים איתו
js/api.js	הקובץ של היום. הוא עובר ל- <code>ts/</code> , והוא נמחק מ- <code>js/</code> בידיים? לא — קרא את הערה למטה
ts/types.ts	שלד. הישות, האיחודים, והשומר
ts/dom.ts	שלד. שני עוזרים גנריים. חלק ד
tsconfig.json	נתון ומלא . לא נוגעים בו
specs/ts-migration.md	הקובץ הראשון שאתה פותח היום. חלק א
js/ שאר	נשאר JavaScript. לא נוגעים בו — חץ משורת ייבוא אחת בחלק ד

tsconfig.json מגדיר "rootDir": "ts" ו-"outDir": "js". ברגע שתיצור ts/api.ts ותריץ את המהדר, הוא יכתוב js/api.js בעצמו – ויחליף את הקובץ הישן. זה מכון: זה אותו קובץ, ולכן אותו נתיב, ולכן render.js ו-events.js לא צריכים לדעת שקרה משהו.

פתח את הפלט אחרי הבנייה הראשונה והסתכל בו. זו החוויה שכל השיעור מסתמך עליה.

מגבלות – מה אסור השבוע

any לא מופיע בשום קובץ ב-ts/. לא כפתרון זמני, לא ב-catch, לא באף מקום. unknown מותר, רצוי, וממש לא אותו דבר.

מה שיוצא מ-res.json() מוקלד **unknown** ולא ב-as X. await res.json()) מהדר, קצר יותר, ובודק בדיוק כלום. זו השורה שהמטלה הזאת קיימת בשבילה.

strict נשאר **true**. אם כיבית אותו כדי שיעבור – לא העברת, ויתרת.

טיפוס שמתאר את הסכמה של ה-API שלך מוצהר ב-ts/api.ts ואינו מיוצא. אף קובץ אחר ביישום לא אמור להכיר את שמות השדות של מישור אחר.

אין שינוי התנהגות. לא הודעה, לא תנאי, לא סדר. אם שיפרת משהו בדרך – החזר אותו.

הדפדפן טוען את js/, לא את ts/. אל תשנה אף <script src>.

חלק א · המפרט, שנכתב ראשון · 12 נקודות ליבה

פתח את specs/ts-migration.md לפני שאתה פותח כלי AI. חמישה סעיפים, והם קצרים בכוונה – מפרט שלוקח עשרים דקות לכתוב הפסיק להיות מפרט.

הסעיף	מה הוא מכריח אותך להחליט
1 · מה אני מעביר ומה לא	שהעברה היא הדרגתית, ושאתה יודע איפה היא נעצרת
2 · הטיפוסים שהקוד צריך	interface או type לכל אחד, ולמה דווקא זה
3 · הגבול	איפה נכנס ערך שלא כתבת, ומה מצמצם אותו
4 · מה חייב להישאר זהה	שלוש התנהגויות, ואיך תבדוק כל אחת
5 · הבקשה, ומה תבדוק בפלט	ארבעה דברים קונקרטיים. "לבדוק שזה עובד" אינו אחד מהם

למה זה נבדק, ולמה זה שווה 12 נקודות

מפרט שנכתב אחרי הקוד תמיד מסכים איתו ולעולם לא מזהיר מפני כלום. הראיה היא ההיסטוריה: **קומיט של המפרט, ואז קומיט של ts/**. בסעיף 4 אתה כותב מה אסור שישתנה, ובסעיף 5 אתה כותב ארבע טעויות שאתה מצפה שהמכונה תעשה – לפני שהיא עשתה אותן. אלה שני הדברים היחידים במטלה שאי אפשר לייצר בדיעבד.

ts/types.ts. כתוב אותו ראשון — כל השאר מחזיר משהו שמתואר כאן.

ב.1 · הצורות

- **Item** — הפריט שלך. העתק את שמות השדות מ- js/items.js; הם כבר קיימים.
- **Suggestion** — הצעה אחרי נרמול. לא מה שה-API שולח.
- **ErrorKind** — ארבע הסיבות לכישלון, כאיחוד של ליטרלים.
- **RequestStatus** — ארבעת המצבים, כאיחוד.

interface או **type**? ההחלטה שלך, ואתה מנמק אותה במפרט. הכלל שעובד: **interface** לצורה של אובייקט שקוד אחר צורך; **type** לאיחוד, לחיתוך, ולכינוי. איחוד לא ניתן לכתוב כ- **interface** בכלל — זו לא העדפה, זו יכולת.

ב.2 · השומר

```
export function isItem(value: unknown): value is Item;
```

המילה **is** היא כל ההבדל. **boolean** אומר "התשובה כן"; **value is Item** אומר "ומכאן והלאה המהדר רשאי להתייחס אליו כ- **Item**". זה הגשר היחיד מ- **unknown** לטיפוס אמיתי שאינו שקר.

קפדן על מה שבלעדיו הציור נשבר, מתירני על מה שמוצג בתנאי. שומר שפוסל פריט שלם בגלל שדה אופציונלי חסר זרק נתונים שהמשתמש היה יכול להשתמש בהם.

למה שומר בכלל, אם יש טיפוסים

הטיפוסים נמחקים בקומפילציה. פתח את js/types.js אחרי הבנייה הראשונה ותספור מה נשאר: השומרים, ושום דבר אחר. **Item** מתאר מה אמור להיות ב- **localStorage**; השומר הוא הקוד היחיד שמגלה מה באמת יש שם. הערך שיצא מהדיסק נכתב על ידי גרסה קודמת של הקוד שלך, או על ידי מישהו עם קונסולה פתוחה.

js/api.js עובר להיות ts/api.ts. אותו קובץ, אותם תנאים, אותו סדר.

מה	איך
חתימות מוקלדות על כל פונקציה מיוצאת	פרמטרים וטיפוס החזרה. אל תיתן להסקה לעשות את זה בשקט
שדות ApiError מוצהרים	<code>readonly kind: ErrorKind</code> — ה-JSDoc היה תגובה, זה נבדק
<code>res.json()</code> מוקלד unknown	השורה שהכול תלוי בה. לא as
הסכמה של ה-API שלך מוצהרת ומקומית	<code>interface</code> , ב- <code>api.ts</code> , בלי <code>export</code>
<code>catch (error)</code> הוא unknown תחת strict	<code>Error instanceof error</code> לפני <code>error.name</code> . זה לא טקס, זה הבדיקה

המלכודת שתפגוש, והיא לא בקוד שלך

```
res.json() מוגדר בטיפוסי ה-DOM כמחזיר Promise<any> — any, לא unknown. כלומר בלי הצהרה מפורשת, payload.query.pages[0].title.toUpperCase() עובר את המהדר בלי מילה, על תשובה של 404.  
let payload: unknown; היא שורה אחת, והיא מחזירה את הגבול למקום.
```

מה שבדקים בסוף, ואתה יכול לבדוק בעצמך

```
npx tsc -p tsconfig.json  
git diff --stat js/api.js
```

הראשונה חייבת לעבור, עם **strict** דלוק. השנייה משווה את הפלט שנוצר לגרסה של שבוע שעבר: סוגריים ורווחים — בסדר; תנאי שהשתנה — לא.

שלוש ההתנהגויות שחייבות להישאר: חיפוש מוצלח מגיע למסך 404 מגיע כשגיאה ולא כתוצאות · הקלדה מהירה לא מאדימה את הפאנל אף פעם.

חלק ד · להקליד את ה-16 DOM נקודות הרחבה

ts/dom.ts. הגנריקה היחידה בקורס, והיא שווה את הזמן.

document.querySelector('#q') הוא **Element | null**, ושני החצאים בדרך: ל-**Element** אין **value**, ו-**null** | נכון בכל שאילתה — כלומר ארבעים ענפי **null** שאיש לא כותב.

התיקון השגוי הוא **HTMLInputElement** as בכל מקום: הוא מכבה את המהדר בדיוק בגבול שבו הוא היה שווה משהו, בארבעים מקומות שכבר אי אפשר למצוא.

```
export function {{CONTENT}}<T extends Element = Element>(selector: string, root?: ParentNode): T;
```

<T extends Element = Element> נקרא: **T** הוא סוג אלמנט כלשהו, מי שקורא בוחר איזה, וברירת המחדל היא **Element** — וזה מה ששומר על **('main')** עובד בלי ארגומנט טיפוס.

ואז חבר אותו. קובץ JavaScript אחד לפחות מייבא מ-**./dom.js**. ומשתמש בו. עוזר שנכתב ולא נקרא הוא קובץ, לא ארגון מחדש — ויש על זה שורה נפרדת בניקוד.

חלק ה · ביקורת על מה שהמכונה כתבה · 12 נקודות אתגר

רשום ב-**PROMPTS.md** שני דברים לפחות שהכלי כתב ושינית, עם הסיבה. לפחות אחד מהם צריך להיות אחד משלושת אלה — כי שלושם יופיעו:

1. **catch (error: any)**. זה התיקון הראשון שהוא מציע כשהוא נתקל ב-**error.name** תחת **strict**, והוא מכבה בדיוק את מה שביקשת. התיקון הנכון הוא **error instanceof Error**.
2. **as על התשובה**. **(await res.json()) as WikiPayload** — מהדר, קצר, בודק כלום.

3. `async function getJson<T>(url: string): Promise<T>`. עוטף `fetch` גנרי, נראה מקצועי מאוד, ואין `T` בזמן ריצה. הוא מבטיח טיפוס שהוא לעולם לא בודק, אז כל מי שקורא לו מקבל אובייקט מוקלד לחלוטין שעשוי להיות מסמך שגיאה.

הגרסה הכנה של 3, כי זו לא "אין להשתמש בגנריקה"

```
async function getJson<T>(url: string, guard: (v: unknown) => v is T): Promise<T>;
```

אותן שלוש שורות, פרמטר אחד נוסף, והקורא מספק גם את הטיפוס וגם את ההוכחה. זה ההבדל בין מערכת טיפוסים לבין קישוט.

מדיניות AI לשבוע הזה

שותף מימוש, כמו משבוע 7, ועם אזהרה ספציפית מאוד לשבוע הזה.

העברה ל-TypeScript היא הדבר שכלי AI עושה הכי מהר והכי משכנע, והכי קל לא לשים לב שהוא ויתר. הפלט מהדר, הוא צבעוני בעורך, ויש בו שלושה מקומות שבהם המהדר כובה בשקט: `any`, `as`, וגנריקה שמבטיחה טיפוס שהיא לא בודקת.

הבקשה שעובדת אומרת מראש מה אסור: "תחת `strict`, בלי `any` בשום מקום, מה שיוצא מ- `res.json()` מוקלד `unknown`, ובלי `as` על ערך שהגיע מהרשת"

ואז קרא את הפלט מול המפרט שכתבת, סעיף 5, שורה-שורה. זו לא פורמליות: אתה כתבת שם ארבעה דברים לחפש לפני שראית את התשובה, וזה ההבדל בין ביקורת לבין הסכמה.

[free-tier!] **בלי מנוי?** אותו מסלול בדיוק, בצ'אט: הדבק את `js/api.js` המלא, הדבק את סעיפים 1-5 של המפרט שכתבת, ובקש קובץ אחד שלם — לא דיף. העתק אותו ל- `ts/api.ts`, הרץ `npx tsc`, וקרא את השגיאות. **שלב הביקורת הזה לחלוטין**, והוא זה ששווה נקודות.

הגשה

תיקיה /week-12 במאגר הסמסטר שלך. **הגשה במודל: כתובת המאגר + ה-SHA של הקומיט.**

הקומיטים חשובים כאן במיוחד: המפרט בקומיט אחד, `ts/` בקומיט שאחריו. זו הראיה היחידה שהמפרט נכתב ראשון, והיא שווה נקודות בחלק א.

אחרי ההגשה, העתק את `ts/` ואת `tsconfig.json` גם ל- `project/` שלך. **הפרויקט מוגש בסוף השבוע הזה**, ודרישות 22-24 שלו הן בדיוק המטלה הזאת.

הגשה מאוחרת: ההגשה נסגרת במועד שכתוב במודל. הגשה שמגיעה אחרי המועד מתקבלת ונבדקת במלואה, ומסומנת כמאוחרת — היא אינה מורידה נקודות.

סיימת כשכל אלה נכונים

- ☐ . `npx tsc -p tsconfig.json` עובר, ו-`strict` עדיין `true`.
- ☐ . פתחת את `js/types.js` שנוצר וספרת מה נשאר בו. **רק השומרים.**

- ☐. חיפשת `any` בכל `ts/` . אפס תוצאות.
- ☐. מחקת בכוונה שדה מ- `isItem` והרצת — ראית שהמהדר **לא** אמר כלום, כי זה זמן ריצה.
- ☐. כתבת `const x: string = 5` איפשהו, ראית את השגיאה, ומחקת. **תדע איך היא נראית.**
- ☐. הקלדת מילה בחיבת החיפוש וקיבלת תוצאות. **בדיוק כמו בשבוע שעבר.**
- ☐. כיבית את ה-Wi-Fi והקלדת. יש משפט, ויש כפתור.
- ☐. הקלדת מילה ארוכה מהר. הפאנל **לא האדים** אף פעם.
- ☐. `git diff` על `js/api.js` מול שבוע שעבר: סוגריים ורווחים, לא תנאים.
- ☐. אף `<script src>` ב-HTML לא השתנה.